Jenkins en Máquina local

Alumno: Yamil Motta

Asignatura: Proyecto Final Grado

Ciclo: 2ºASIR

Curso: 25/26

Introducción de Jenkins 3

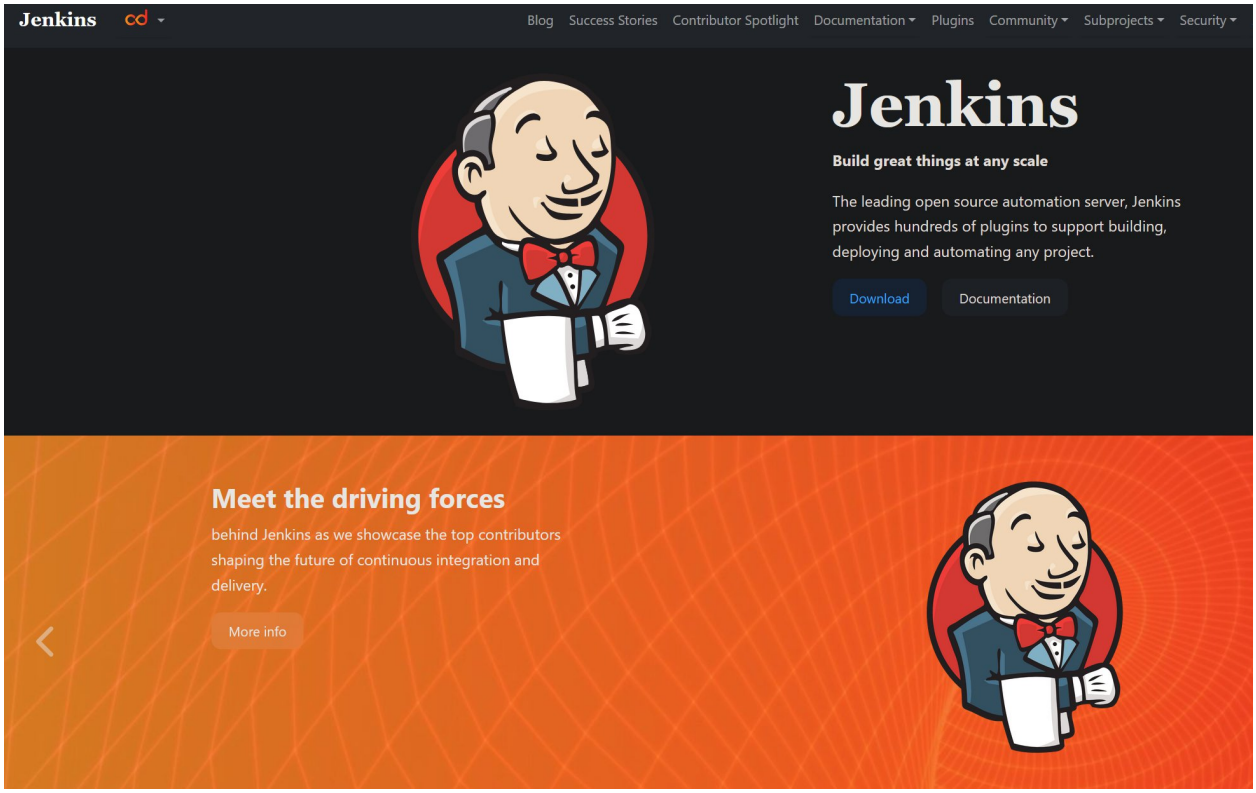
Fases (plan de ruta provisional) 3
Ubuntu Server en VMWare 5
Instalación del motor de contenedores 8
Gestión de permisos y verificación de motor 9
Despliegue de Jenkins 10

Persistencia de Datos y Docker-in-Docker (DinD) 10
Arquitectura de ejecución: Docker-out-of-Docker (DooD) 10

Introducción de Jenkins

Jenkins es una herramienta de automatización de código abierto líder en el sector, diseñada para facilitar la Integración Continua y la Entrega Continua (CI/CD). Actúa como un servidor central que permite a los equipos de desarrollo automatizar las distintas etapas del ciclo de vida del software, como la compilación, las pruebas y el despliegue de aplicaciones. Su gran potencia reside en su ecosistema de más de 1.800 complementos (plugins), lo que le permite integrarse con prácticamente cualquier herramienta del flujo de trabajo, ayudando a detectar errores rápidamente y a entregar software de manera más ágil y eficiente.

Página oficial: https://www.jenkins.io/

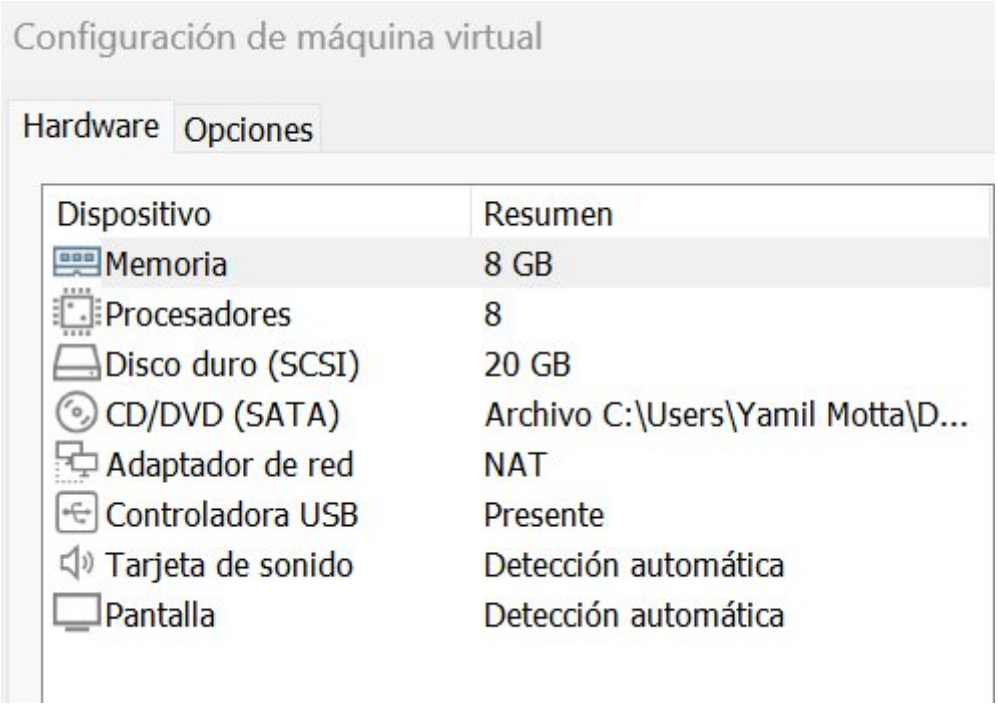


Ubuntu Server en VMWare



Nombre del servidor: g1server-cicd
Usuario: yamil

Propiedades de máquina virtual:



Ubuntu Server IP Local: 172.16.18.131/24

```
yamil@g1server-cicd:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 00:0c:29:97:7e:ec brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 172.16.18.131/24 metric 100 brd 172.16.18.255 scope global dynamic ens33
       valid_lft 1674sec preferred_lft 1674sec
    inet6 fe80::20c:29ff:fe97:7eec/64 scope link
       valid_lft forever preferred_lft forever
yamil@g1server-cicd:~$ _
```

Creación de clave SSH para conexión con el server:

```
PS C:\WINDOWS\system32> ssh-keygen -t ed25519 -C "jenkins-tfg"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\Yamil Motta/.ssh/id_ed25519):
C:\Users\Yamil Motta/.ssh/id_ed25519 already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\Yamil Motta/.ssh/id_ed25519
Your public key has been saved in C:\Users\Yamil Motta/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:jWMm7CFk4bfsRcKa2bPJ1d0UsehuaVN2QggzR8Ts6XU jenkins-tfg
The key's randomart image is:
+--[ED25519 256]--+
|    .     +=+ o. |
|   . o     =oo o |
|    + + .  .o.+  |
|   o O + + oo+. E|
|    = O S o.o.=..|
|     = @ . ..+ o |
|      *     *    |
|           o .   |
|                 |
+----[SHA256]-----+
PS C:\WINDOWS\system32>
```

Añadir la SSH Key a known hosts para que cada vez que entre no me pida contraseña:

```
PS C:\WINDOWS\system32> Get-Content ~/.ssh/id_ed25519.pub | ssh yamil@172.16.18.131 "mkdir -p ~/.ssh && cat >> ~/.ssh/au
thorized_keys"
The authenticity of host '172.16.18.131 (172.16.18.131)' can't be established.
ED25519 key fingerprint is SHA256:YsLmD7VBg8R/85R3TQCIvOU07gwN1itt+P1duqNMySw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.16.18.131' (ED25519) to the list of known hosts.
yamil@172.16.18.131's password:
PS C:\WINDOWS\system32>
```

Ahora, el server es accesible por SSH con clave:

```
~\Downloads
ssh yamil@172.16.18.131
Enter passphrase for key 'C:\Users\Yamil Motta/.ssh/id_ed25519':
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-106-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of mié 18 mar 2026 15:45:11 UTC

  System load:  0.08              Processes:             270
  Usage of /:   31.3% of 9.75GB   Users logged in:       1
  Memory usage: 5%                IPv4 address for ens33: 172.16.18.131
  Swap usage:   0%


El mantenimiento de seguridad expandido para Applications está desactivado

Se pueden aplicar 9 actualizaciones de forma inmediata.
1 de estas es una actualización de seguridad estándar.
Para ver estas actualizaciones adicionales, ejecute: apt list --upgradable

Active ESM Apps para recibir futuras actualizaciones de seguridad adicionales.
Vea https://ubuntu.com/esm o ejecute «sudo pro status»


Last login: Wed Mar 18 15:03:21 2026 from 172.16.18.1
yamil@g1server-cicd:~$
```

En ubuntu server, se prepara el entorno para Docker. Preparamos las dependencias seguras.

ca-certificates permite al sistema operativo validar los certificados SSL/TLS de las webs a las que se conecta. curl es la herramienta para descargar archivos desde la terminal. El último comando simplemente crea una carpeta (keyrings) con los permisos estrictos adecuados (0755) para guardar claves de seguridad.

```
yamil@g1server-cicd:~$ sudo install -m 0755 -d /etc/apt/keyrings
[sudo] password for yamil:
```

Descargamos la clave GPG oficial de Docker y le damos permisos de lectura:

```
yamil@g1server-cicd:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
yamil@g1server-cicd:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
yamil@g1server-cicd:~$ cd /etc/apt/keyrings/
yamil@g1server-cicd:/etc/apt/keyrings$ ls -l
total 4
-rw-r--r-- 1 root root 3817 mar 18 15:06 docker.asc
yamil@g1server-cicd:/etc/apt/keyrings$
```

Esta clave sirve para que el gestor de paquetes (apt) pueda verificar que el software de Docker que vamos a descargar proviene realmente de sus creadores.

```
yamil@g1server-cicd:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(. /etc/os-releas
e && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

En lugar de instalar la versión de Docker que viene por defecto en los repositorios de Ubuntu (que suele estar obsoleta), este comando apunta directamente a los servidores de Docker. Esto garantiza tener siempre la última versión estable, con los últimos parches de seguridad aplicados.

Instalación del motor de contenedores

Se instalan varios componentes modulares:

- docker-ce: El motor principal (el daemon que corre en segundo plano).
- docker-ce-cli: La interfaz de comandos que usas tú para interactuar con él.
- containerd.io: El entorno de ejecución de bajo nivel que gestiona el ciclo de vida de los contenedores.
- docker-buildx-plugin es un componente que extiende las capacidades de construcción (build) de Docker. En la instalación del motor, incluí este plugin oficial porque proporciona herramientas avanzadas para la creación de imágenes
- docker-compose-plugin: La herramienta clave que nos permitirá definir toda nuestra infraestructura de CI/CD (Jenkins, túneles, etc.) en un solo archivo YAML.

```
yamil@g1server-cicd:~$ sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  docker-ce-rootless-extras libslirp0 pigz slirp4netns
Paquetes sugeridos:
  cgroupfs-mount | cgroup-lite docker-model-plugin
Se instalarán los siguientes paquetes NUEVOS:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin libslirp0 pigz slirp4netns
0 actualizados, 9 nuevos se instalarán, 0 para eliminar y 10 no actualizados.
Se necesita descargar 97,2 MB de archivos.
Se utilizarán 393 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] s
Des:1 https://download.docker.com/linux/ubuntu noble/stable amd64 containerd.io amd64 2.2.2-1~ubuntu.24.04~noble [23,6 MB]
Des:2 http://archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65,6 kB]
Des:3 http://archive.ubuntu.com/ubuntu noble/main amd64 libslirp0 amd64 4.7.0-1ubuntu3 [63,8 kB]
Des:4 http://archive.ubuntu.com/ubuntu noble/universe amd64 slirp4netns amd64 1.2.1-1build2 [34,9 kB]
10% [1 containerd.io 3.325 kB/23,6 MB 14%]
```

Gestión de permisos y verificación de motor

```
yamil@g1server-cicd:~$ sudo usermod -aG docker $USER
yamil@g1server-cicd:~$
```

Añado mi usuario estándar al grupo de administradores de Docker. El motor de Docker necesita privilegios elevados para manipular redes y procesos del sistema. Por defecto, solo el superusuario (root) puede usarlo. Añadir mi usuario a este grupo mejora la agilidad operativa, evitando tener que usar sudo para cada comando diario, manteniendo un buen equilibrio en el modelo de permisos.

Probamos una imagen sencilla de docker para confirmar el correcto funcionamiento:

```
yamil@g1server-cicd:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
ea52d2000f90: Download complete
Digest: sha256:85404b3c53951c3ff5d40de0972b1bb21fafa2e8daa235355baf44f33db9dbdd
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

yamil@g1server-cicd:~$
```

Al lanzar el comando, el motor comprobó que no tenía la imagen en local, la descargó del registro público (Docker Hub) y la instanció en un contenedor aislado. Esto confirmó que el daemon está operando correctamente, que la VM tiene resolución DNS y salida a internet, y que los permisos están bien configurados para operar sin elevar privilegios (sudo).

Despliegue de Jenkins

Persistencia de Datos y Docker-in-Docker (DinD)

Archivo docker-compose.yml

```
 GNU nano 7.2                                                docker-compose.yml
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins_core
    user: root
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - ./jenkins_data:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    restart: unless-stopped
```

Utilizando Docker Compose, centralizo la configuración del servicio de integración continua. El archivo .yml define el servicio principal (jenkins_core) basándose en la imagen oficial con soporte a largo plazo (lts). He configurado el mapeo de puertos directos (8080 para la interfaz web y 50000 para los agentes) y he establecido una política de reinicio automático (unless-stopped) para garantizar la alta disponibilidad del servicio ante reinicios de la máquina virtual.

La sección volumes del archivo docker-compose.yml vincula carpetas del sistema host con el interior del contenedor.

Los contenedores son efímeros por naturaleza; si se destruyen, los datos se pierden. Para solucionar esto en un entorno de producción, mapeé un volumen local (./jenkins_data) hacia el directorio interno /var/jenkins_home. Esto garantiza que los pipelines, configuraciones y el historial de builds persistan en el disco de mi máquina virtual independientemente del ciclo de vida del contenedor. Adicionalmente, monté el socket de Docker (/var/run/docker.sock) dentro de Jenkins. Esta técnica es crítica: permite que Jenkins se comunique directamente con el motor Docker del servidor anfitrión, otorgándole la capacidad de construir y ejecutar otros contenedores durante la ejecución de las tuberías CI/CD.

Arquitectura de ejecución: Docker-out-of-Docker (DooD)

Al estar Jenkins encapsulado dentro de un contenedor, por defecto no puede ver ni tener permisos sobre el sistema anfitrión (la máquina virtual) para instanciar nuevos contenedores durante las fases de build y despliegue.

Para resolver este aislamiento sin comprometer el rendimiento ni recurrir a anidar contenedores dentro de contenedores, implementé un enfoque Docker-out-of-Docker. Esto se logró montando el archivo /var/run/docker.sock del host directamente en el contenedor de Jenkins. Este archivo es el socket UNIX a través del cual el cliente de Docker se comunica con su propio motor (el daemon). Al compartir este conducto, cuando el pipeline de Jenkins ejecuta un comando como docker run, la orden no se procesa dentro del contenedor de Jenkins, sino que se delega al motor principal de la máquina virtual (Ubuntu Server). Como resultado, los contenedores generados por Jenkins se ejecutan como "hermanos" (siblings) en el host, aprovechando los recursos de forma eficiente y segura.